

CSCE 645 GEOMETRIC MODELING · MODULE 08 · LECTURE NOTES

The Laplacian and Smoothing

One operator, and most of geometry processing.

MODULE OUTCOMES

- Build the cotangent Laplacian as a sparse matrix.
- Implement explicit and implicit Laplacian smoothing.
- Explain the stability limit of explicit integration.
- Explain shrinkage and how to prevent it.
- Explain why the Laplacian underlies so many algorithms.

1 What the Laplacian is

$$(\Delta \mathbf{f})_{\mathbf{i}} = \sum_{\mathbf{j} \in \mathbf{N}(\mathbf{i})} w_{\mathbf{ij}} (\mathbf{f}_{\mathbf{j}} - \mathbf{f}_{\mathbf{i}})$$

The Laplacian at vertex $\mathbf{i}$ measures how far the value there is from a weighted average of its neighbours. If $\mathbf{f}$ is already the average of its surroundings, $\Delta \mathbf{f} = 0$. The operator detects local deviation from smoothness, and that single description accounts for everything it is used for.

Weighting	$w_{\mathbf{ij}}$	Behaviour
Uniform (graph Laplacian)	1, or $1/\deg(\mathbf{i})$	Depends only on connectivity, so it ignores the actual geometry. Smoothing with it redistributes vertices by valence rather than by shape. Useful for pure connectivity work; wrong for geometry.
Cotangent	$\frac{1}{2}(\cot \alpha_{\mathbf{ij}} + \cot \beta_{\mathbf{ij}})$	Derived from a finite-element discretisation (Module 07). Geometry-aware and the correct choice for everything in this course.

WHY ONE OPERATOR SERVES SO MANY PROBLEMS

$\Delta \mathbf{x} = 2\mathbf{H}\mathbf{n}$. The Laplacian of the position function is mean curvature times the normal (Module 07). So flowing along $-\Delta \mathbf{x}$ is mean curvature flow, and smoothing a surface is literally reducing its curvature.

$\Delta \mathbf{f} = \mathbf{0}$ defines harmonic functions — the smoothest function consistent with given boundary values. That is exactly the parameterisation problem of Module 09.

Δ is the gradient of the Dirichlet energy $\int |\nabla \mathbf{f}|^2$. Any problem phrased as 'find the function that is as smooth as possible subject to constraints' leads to a Laplace system. Deformation (Module 13) is this.

The eigenvectors of Δ form a Fourier basis on the surface. Spectral filtering, shape descriptors, segmentation, and compression all follow.

Four unrelated-looking problems, one sparse matrix. It is worth building carefully.

1.1 Building it

```
std::vector<Eigen::Triplet<double>> T;
for (Edge e : mesh.edges()) {
    int i = e.v0(), j = e.v1();
    double w = 0.5 * (cot(e.alpha()) + cot(e.beta()));
    T.emplace_back(i, j, w);
    T.emplace_back(j, i, w);
    T.emplace_back(i, i, -w);
    T.emplace_back(j, j, -w);
}
L.setFromTriplets(T.begin(), T.end());
```

ASSERT THAT ROWS SUM TO ZERO

Each edge contributes $+w$ to two off-diagonal entries and $-w$ to two diagonal entries, so every row sums to exactly zero by construction.

The property matters: a zero row sum means L annihilates constant functions, so a flat region has zero Laplacian and a rigid translation of the mesh does not change it.

If your rows do not sum to zero, a constant function has non-zero Laplacian, smoothing will drift, and every system you solve will be subtly wrong. Assert it immediately after construction — it is the cheapest correctness check available.

L is symmetric and sparse, with roughly 7 non-zeros per row given average valence 6 (Module 01). Use a sparse matrix type; a dense $100,000 \times 100,000$ matrix is 80 GB.

2 Smoothing

Move each vertex toward the average of its neighbours. Since $\Delta \mathbf{x} = 2\mathbf{H}\mathbf{n}$, this is mean curvature flow: a geometric process with a precise meaning, not an ad hoc averaging.

2.1 Explicit integration

$$\mathbf{x}^{n+1} = \mathbf{x}^n + \lambda \Delta \mathbf{x}^n$$

One sparse matrix-vector product per step. Trivial to implement, and **conditionally stable**.

THE STABILITY LIMIT, AND WHY IT HURTS

This is forward Euler applied to the heat equation, and forward Euler on a diffusion problem is stable only when λ is roughly below h^2 , where h is the *smallest* edge length in the mesh.

Exceed it and the iteration oscillates with growing amplitude and the mesh explodes — a distinctive and memorable failure.

The practical problem is the word 'smallest'. A single small triangle anywhere forces a tiny time step for the entire mesh, so a mesh with one bad triangle smooths hundreds of times more slowly than it should.

Run this deliberately in the exercises. Finding the λ at which your mesh explodes, and relating it to your shortest edge, makes the constraint concrete.

2.2 Implicit integration

$$(\mathbf{I} - \lambda \Delta) \mathbf{x}^{n+1} = \mathbf{x}^n$$

Backward Euler. Each step requires solving a sparse linear system rather than performing a multiplication, and in exchange the scheme is **unconditionally stable**: any λ produces a sensible result, so a single large step can accomplish what would take thousands of explicit steps.

FACTOR ONCE, SOLVE MANY TIMES

$(\mathbf{I} - \lambda \Delta)$ is symmetric and positive definite for $\lambda > 0$, so it admits a sparse Cholesky factorisation.

Compute that factorisation **once**; each subsequent solve is a cheap forward and back substitution. If the mesh connectivity and λ do not change, the factorisation is reusable across every step and across all three coordinate columns.

This pattern — an expensive factorisation amortised over many cheap solves — recurs in parameterisation (Module 09) and deformation (Module 13), where it is what makes interactive editing possible. It is the single most important performance idea in geometry processing.

3 Shrinkage

Mean curvature flow shrinks closed surfaces. A sphere under mean curvature flow contracts and vanishes in finite time. Since Laplacian smoothing is mean curvature flow, smoothing a closed mesh shrinks it — correct behaviour, and almost never what you wanted.

Fix	Method	Trade-off
Volume preservation	After each step, compute the enclosed volume and uniformly rescale to restore the original.	Simple and effective globally; does not prevent local shrinkage of features.
Taubin $\lambda \mu$	Alternate a smoothing step with $\lambda > 0$ and an <i>unsmoothing</i> step with $\mu < 0$ and $ \mu $ slightly greater than λ .	Acts as a genuine low-pass filter with a near-unit passband, so low frequencies are preserved and shrinkage essentially vanishes. Two cheap explicit steps, no linear solve. Usually the right answer.
Tangential smoothing	Project the Laplacian update onto the tangent plane before applying it.	Vertices redistribute across the surface without moving it, improving mesh quality with no shape change at all. Useful as a remeshing step (Module 10).

4 Feature-preserving smoothing

A LOW-PASS FILTER CANNOT DISTINGUISH NOISE FROM A CREASE

Scanner noise and a genuine sharp edge are both high-frequency content. Laplacian smoothing removes both, so cleaning a scanned mechanical part rounds off every edge the part actually has.

Bilateral filtering weights each neighbour by two factors: spatial proximity, as usual, and *normal similarity*. A neighbour whose normal differs sharply — one across a crease — receives almost no weight, so the filter averages along features and not across them.

This is precisely the bilateral filter from image processing, transplanted to a surface, and the same idea appears again in CSCE 748 for denoising photographs. Edge-aware filtering is one idea with many domains.

Method	Removes noise	Preserves features	Cost
Explicit Laplacian	Yes	No	Cheap per step; many steps needed.
Implicit Laplacian	Yes	No	One factorisation, then cheap. Large steps allowed.
Taubin $\lambda \mu$	Yes	No, but no shrinkage	Two explicit steps. No solve.
Bilateral	Yes	Yes	Several passes; more expensive per pass.

Method	Removes noise	Preserves features	Cost
Tangential only	No — redistributes vertices	Shape unchanged	Cheap. A mesh-quality tool rather than a denoiser.

5 What this matrix does next

Module	Problem	System solved
09 Parameterisation	Flatten a surface patch to the plane with least distortion.	$\Delta u = 0$ with fixed boundary — harmonic maps; or the least-squares conformal system.
13 Deformation	Move a handle while preserving local detail everywhere else.	Minimise $\ \Delta x - \delta\ ^2$ subject to handle constraints — a Laplace system with soft constraints.
Spectral geometry	Shape descriptors, segmentation, compression, correspondence.	Eigenvectors of Δ , which form a Fourier basis on the surface.
Geodesic distance	Distance from a source across the surface.	The heat method: one heat diffusion solve, then one Poisson solve. Two systems with the same matrix you already factored.

It is not an overstatement to say the Laplacian is most of geometry processing. Build it correctly, assert the zero row sum, keep the factorisation around, and the remaining modules become applications of work you have already done.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Keenan Crane — DDG, the Laplace operator and smoothing lectures	The operator derived from exterior calculus, with smoothing as curvature flow.
Desbrun, Meyer, Schröder & Barr — Implicit Fairing of Irregular Meshes (free)	The paper that introduced implicit smoothing and the cotangent weights to graphics. Still the clearest account of the stability argument.
Taubin — A signal processing approach to fair surface design (free)	The $\lambda \mu$ filter, derived properly as a signal-processing problem.
libigl tutorial — Laplacian and smoothing	Working code with Eigen, including the sparse solve pattern.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Build the cotangent Laplacian as a sparse matrix. Assert that every row sums to zero and that the matrix is symmetric.
- 2 Verify $\Delta x = 2Hn$ numerically: compute the Laplacian of position and compare against the mean curvature normal from Module 07.
- 3 Implement explicit smoothing. Increase λ until the mesh explodes, record the threshold, and compare it against the square of your shortest edge length.
- 4 Implement implicit smoothing with a sparse Cholesky factorisation. Confirm it is stable at λ values a thousand times larger than the explicit limit, and measure the speedup for equivalent smoothing.
- 5 Measure shrinkage: smooth a unit sphere and plot enclosed volume against iteration count. Then implement Taubin $\lambda|\mu$ and plot it on the same axes.
- 6 Implement bilateral mesh smoothing. Apply both it and plain Laplacian smoothing to a noisy scan of a mechanical part and capture the difference at the sharp edges.
- 7 Implement tangential-only smoothing and show it improves triangle quality metrics without changing the surface.
- 8 Compute the first ten eigenvectors of L on a simple mesh and visualise them as per-vertex colour. Observe that they look like vibration modes.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does the Laplacian measure, in one sentence?
- 2 Why use cotangent weights rather than uniform ones?
- 3 Why must the rows of L sum to zero, and what goes wrong if they do not?
- 4 State the stability condition for explicit smoothing and explain why one small triangle is a problem.
- 5 What does 'factor once, solve many times' mean and why does it matter?
- 6 Why does Laplacian smoothing shrink a closed surface, and how does Taubin $\lambda|\mu$ avoid it?
- 7 Why can Laplacian smoothing not preserve sharp features, and what does bilateral filtering change?
- 8 Name three other problems that reduce to solving a system with this same matrix.